

Plano de implementação do ACMEPolling

Este documento organiza o desenvolvimento do Trabalho I de Fundamentos de Desenvolvimento de Software. O plano parte do estado atual do repositório e avança até a entrega, indicando o percentual acumulado, as atividades e as formas de validação.

Visão geral

Progresso	Marco	Resultado esperado
20%	Base existente	Entidades e dados iniciais presentes
25%	Ambiente	Projeto compila com Java 21
40%	Modelo e arquitetura	Voto corrigido, enum e camadas organizadas
63%	Candidatos	Cadastro, consulta, atualização e remoção lógica
70%	Votação	Cadastro e classificação de votos
88%	Apuração	Consultas, vencedor e mais votados
94%	Testes	Casos normais e de erro automatizados
98%	Relatório	UML, arquitetura e padrões documentados
100%	Entrega	Compilação, demonstração e pacote final validados

1 Estado atual 20%

- ☒ Projeto Spring Boot criado.
- ☒ Classes Candidato, Partido, Localidade e Voto.
- ☒ Acervo para armazenamento em memória.
- ☒ Três partidos, três localidades e três candidatos elegíveis.
- ☒ Dois candidatos na mesma localidade.
- ☒ Validação de voto iniciada.
- ☐ Compilação validada com Java 21.
- ☐ Dez endpoints do enunciado.
- ☐ Testes completos.
- ☐ Relatório e diagramas.

2 Preparar o ambiente 25%

Atividades

- ☐ Instalar ou selecionar o JDK 21.
- ☐ Configurar JAVA_HOME.
- ☐ Confirmar as versões de java e javac.
- ☐ Executar a compilação e o teste inicial.

Validação

```
java -version
javac -version
cd E:\igor\Pucrs\FDS-Trab1-PabloVinnicios-IgorXavier\demo
.\mvnw.cmd test
```

Os dois primeiros comandos devem mostrar Java 21 e o Maven deve terminar com BUILD SUCCESS. Não avance enquanto o projeto não compilar, pois o enunciado informa que erros de compilação zeram o trabalho.

3 Corrigir o modelo e organizar a arquitetura 40%

Corrigir Voto 30%

- ☐ Alterar o construtor para receber candidato e localidade.
- ☐ Adicionar o campo valido.
- ☐ Opcionalmente, registrar motivoInvalidacao.
- ☐ Garantir que votos inválidos sejam armazenados para permitir sua contagem.

```
public Voto(int id, int hora, Candidato candidato, Localidade localidade) {
    this.id = id;
    this.hora = hora;
    this.candidato = candidato;
    this.localidade = localidade;
}
```

Teste criando um voto e confirmando que candidato e localidade não ficam nulos.

Criar a situação do candidato 35%

- ☐ Criar SituacaoCandidato como enum.
- ☐ Substituir a situação em texto por esse tipo.
- ☐ Fazer todo candidato novo iniciar como PRECANDIDATO.
- ☐ Implementar as transições permitidas.

```
public enum SituacaoCandidato {
    PRECANDIDATO, ELEGIVEL, INELEGIVEL, ELEITO, NAOELEITO, REMOVIDO
}
```

Transições necessárias:

- PRECANDIDATO para ELEGIVEL, INELEGIVEL ou REMOVIDO.
- INELEGIVEL para ELEGIVEL ou REMOVIDO.
- ELEGIVEL para INELEGIVEL, ELEITO ou NAOELEITO.
- REMOVIDO não volta a outro estado.

Organizar as camadas 40%

controller -> service -> repository/Acervo -> model
^
dto para entradas e respostas

- ☐ Criar pacotes controller, service, repository, model e dto.
- ☐ Criar CadastroController, VotacaoController e ApuracaoController.
- ☐ Criar serviços para candidatos, votação e apuração.
- ☐ Manter regras de negócio nos serviços, não nos controllers.
- ☐ Criar DTOs que reproduzam exatamente os JSONs pedidos.

Após reorganizar, execute `.\mvnw.cmd test` novamente.

4 Implementar candidatos 63%

Listar todos 48%

GET /acmepolling/cadastro/listacandidatos

- ☐ Retornar número, nome, situação, nome do partido e nome da localidade.
- ☐ Usar DTO para não expor objetos inteiros no JSON.

```
[{"numero":101,"nome":"Agostine","situacao":"ELEGIVEL","nome_partido":"Partido da Computação","nome_localidade":"Porto Alegre"}]
```

Filtrar por localidade e situação 53%

GET /acmepolling/cadastro/listacandidatoslocalidade/{cep}/situacao/{situacao}

- ☐ Filtrar pelo CEP e pela situação.
- ☐ Retornar apenas número e nome.
- ☐ Testar lista vazia, CEP inexistente e situação inválida.

Cadastrar candidato 58%

POST /acmepolling/cadastro/cadcandidato

```
{"numero":404,"nome":"Carolina","codigo":10,"cep":"92000-000"}
```

- ☐ Rejeitar número repetido e nome vazio.
- ☐ Verificar se partido e localidade existem.
- ☐ Iniciar como PRECANDIDATO.
- ☐ Retornar true ou false.

Atualizar e remover 63%

- PUT /acmepolling/cadastro/atualizacandidato/{numero}/situacao/{status}
- DELETE /acmepolling/cadastro/removecandidato
- ☐ Validar a transição de estado.
- ☐ Retornar o candidato atualizado no PUT.
- ☐ Na remoção, trocar a situação para REMOVIDO sem excluir da lista.
- ☐ Retornar false quando o candidato não existir.

5 Implementar a votação 70%

POST /acmepolling/votacao/cadvoto

```
{"id":1,"hora":10,"numero":101,"cep":"90000-000"}
```

- ☐ Localizar candidato e localidade.
- ☐ Rejeitar ID repetido.
- ☐ Classificar como inválido quando estiver fora de 8h a 17h.
- ☐ Classificar como inválido quando o candidato não existir ou não for elegível.
- ☐ Classificar como inválido quando candidato e voto forem de localidades diferentes.
- ☐ Classificar como inválido quando o limite de eleitores tiver sido atingido.
- ☐ Armazenar votos válidos e inválidos.

O limite deve ser verificado pela quantidade de votos já registrados na localidade, e não por `indexOf`.

Testes manuais: votos às 8h e 17h; votos às 7h e 18h; candidato inexistente; candidato inelegível; CEP diferente; limite excedido; ID repetido.

6 Implementar a apuração 88%

Votos de um candidato 75%

GET /acmepolling/apuracao/consultacandidato?numero=101

- ☐ Contar votos válidos.
- ☐ Contar votos inválidos.
- ☐ Definir resposta para candidato inexistente.

Votação da localidade 80%

GET /acmepolling/apuracao/consultalocalidade?cep=90000-000

- ☐ Retornar todos os candidatos da localidade.
- ☐ Informar as duas contagens de cada candidato.
- ☐ Comparar o JSON com uma soma manual dos votos de teste.

Eleito da localidade 85%

GET /acmepolling/apuracao/consultaeleito?cep=90000-000

- ☐ Considerar apenas candidatos elegíveis e votos válidos.
- ☐ Escolher a maior quantidade de votos.
- ☐ Em empate, escolher quem recebeu o último voto mais cedo.
- ☐ Atualizar o vencedor para ELEITO e os demais para NAOELEITO.

Teste de empate: candidato 101 recebe votos às 9h e 10h; candidato 202 recebe votos às 8h e 11h. O candidato 101 vence porque seu último voto ocorreu primeiro.

Mais votados por partido 88%

GET /acmepolling/apuracao/listamaisvotadospartido

- ☐ Agrupar candidatos por partido.
- ☐ Contar somente votos válidos.
- ☐ Selecionar o mais votado de cada partido.
- ☐ Definir e testar o comportamento em empate.

7 Criar testes automatizados 94%

Use JUnit 5 e MockMvc.

- ☐ Pelo menos um teste de sucesso para cada endpoint.
- ☐ Pelo menos um teste de erro para cada endpoint.
- ☐ Testes para todas as regras de voto.
- ☐ Testes das transições de situação.
- ☐ Teste do desempate.
- ☐ Teste da remoção lógica.
- ☐ Teste do limite de eleitores.
- ☐ Testes dos formatos JSON.

Meta recomendada: 25 a 35 testes. Execute todos com:

```
.\mvnw.cmd test
```

O resultado deve mostrar zero falhas, zero erros e BUILD SUCCESS.

8 Preparar relatório e diagramas 98%

Diagrama de classes

- ☐ Mostrar entidades, atributos, relacionamentos e multiplicidades.
- ☐ Incluir SituacaoCandidato e as classes adicionadas ao modelo final.

Diagrama de estados

- ☐ Representar todas as situações e transições permitidas.

Relatório

- ☐ Descrever o sistema e sua arquitetura.
- ☐ Inserir os diagramas.
- ☐ Explicar as regras de validação.
- ☐ Indicar onde foram usados arquitetura em camadas, DTO, Repository e outros padrões reais do código.
- ☐ Adicionar exemplos de requisições e evidências de testes.
- ☐ Identificar os integrantes.

9 Validação final e entrega 100%

- ☐ Executar .\mvnw.cmd clean test e confirmar BUILD SUCCESS.
- ☐ Iniciar a aplicação do zero.
- ☐ Executar e conferir os dez endpoints.
- ☐ Confirmar que não existem erros no terminal.
- ☐ Enviar código, testes, relatório e diagramas ao GitHub.
- ☐ Compartilhar o repositório privado com mhyamaguti.
- ☐ Montar o arquivo compactado para o Moodle.
- ☐ Preparar dois computadores se o trabalho for em dupla.
- ☐ Ensaiar a demonstração.

10 Tutorial para executar e testar

Iniciar o projeto

```
cd E:\igor\Pucrs\FDS-Trab1-PabloVinnicios-IgorXavier\demo
java -version
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run
```

Aguarde Started DemoApplication. O servidor estará em `http://localhost:8080`. O terminal ficar ocupado enquanto o servidor estiver rodando; isso é normal.

Abra `http://localhost:8080/`. A resposta atual esperada é `Aplicacao Spring-Boot funcionando!`.

Testar uma consulta

```
Invoke-RestMethod -Method Get `
-Uri "http://localhost:8080/acmepolling/cadastro/listacandidatos"
```

Testar o cadastro de candidato

```
$corpo = @{
    numero = 404
    nome = "Carolina"
    codigo = 10
    cep = "92000-000"
} | ConvertTo-Json

Invoke-RestMethod -Method Post `
-Uri "http://localhost:8080/acmepolling/cadastro/cadcandidate" `
-ContentType "application/json" `
-Body $corpo
```

Parar o servidor

Pressione `Ctrl + C` no terminal em que o Spring Boot está sendo executado.

11 Rotina recomendada de desenvolvimento

Para cada pequena funcionalidade:

1. Salvar os arquivos.
2. Executar `.\mvnw.cmd test`.
3. Iniciar a aplicação.
4. Testar o endpoint alterado.
5. Conferir o JSON e pelo menos um caso de erro.
6. Fazer um commit.
7. Avançar para a próxima atividade.

Exemplos de commits:

```
feat: adiciona enum de situacao do candidato
feat: implementa cadastro de candidatos
feat: implementa validacao e cadastro de votos
feat: implementa apuracao por localidade
test: adiciona testes dos endpoints de votacao
docs: adiciona diagramas e relatorio
```